

MiniGuardian — Security Features Catalog

Compiled from a full source sweep of the MiniGuardian workspace.

This document catalogs every security feature in the codebase — protection, detection, enforcement, **deception**, and **AI containment** — with the module (file) that implements each one. Companion to AGENTS.md, docs/security_assessment.md, docs/AI_WORKSPACE_ENVELOPE_SPEC.md, and docs/SELF_LEARNING_ARCHITECTURE.md. Proprietary — Jayce Automata Research Source License v1.0. View/study/run for personal evaluation only; no redistribution/modification/derivatives.

Table of Contents

1. [What MiniGuardian Is & How the Layers Nest](#)
2. [Core Hardening](#)
3. [Detection & Monitoring](#)
4. [Enforcement & Response](#)
5. [Deception: Active Defense](#)
6. [Trust Engine & Identity](#)
7. [Integrity, Persistence & Recovery](#)
8. [Deterministic Kernel & Proof-Checked Enforcement](#)
9. [AI Containment & AI-Content Safety](#)
10. [Communications, Updates & Sidecars](#)
11. [Fleet Visibility, Forensics & Observability](#)
12. [Robotics Saddle & Actuation Gating](#)
13. [Deployment Modes, Profiles & Hardening Scripts](#)
14. [Cross-Cutting Engineering Contracts](#)
15. [Appendix: Module → Feature Map](#)
16. [The Learning System & Cognition](#)
17. [The Memory System](#)
18. [The Adapters, Bridges & Shims](#)
19. [Clustering & Fleet Topology](#)
20. [The Illusion Shroud](#)
21. [The Kernel Illusion Layer & Frozen ABI](#)
22. [The TRust Cognitive Terminal](#)
23. [Watchdog, Sentinel, Gates, Seal & No Backdoors](#)
24. [The Metrics TUI & Operational Console](#)
25. [How It All Interlocks](#)

26. [The Biggest Illusion: Self-Disguise & Probe-Feed](#)
 27. [The Alien Bridge: Foreign-Sensor Ingestion](#)
-

1. What MiniGuardian Is & How the Layers Nest

MiniGuardian is a **single-host Linux security daemon** built on the deterministic **Jayce kernel** (a Rust `#![no_std]` microkernel-style engine substrate). It is not a classic EDR/HIDS/AV — it is closest to a **host-intrusion-prevention + endpoint-hardening daemon** whose differentiator is a formally deterministic kernel engine model that adjudicates trust and fires security engines on a tick schedule with provable invariants (`kernel_bridge.rs`, `jayce_kernel_src/`).

The protection is layered — the failure of any one layer leaves the others standing (defense-in-depth):

14. Build/engineering contract	static musl, zero warnings, sealed
13. Deployment envelope	seal.sh, profiles, chatter +i, read-only mo
12. Robotics saddle	signed apodeixis missions, fail-closed act
11. Fleet/observability	cosmic map, forensic report, SIEM, audit c
10. Comms/updates	gossip, OTA, sidecar, socket auth
9. AI containment	prompt guard, memory-poison defense, eval
8. Kernel/proof	deterministic engines, Apodeixis missions
7. Integrity/self-repair	shadow backup, host integrity, seal
6. Trust engine	ancestry, baseline, self-identity, kernel
5. Deception	illusion bubble, honeytokens, paranoid mod
4. Enforcement	eBPF, Landlock, kill/quarantine, brain esc
3. Detection	guards + detectors + monitors
2. Core hardening	binary, runtime, deployment self-defense

Scale: the miniguard daemon alone is ~117 modules (~80k LOC), plus `trust_core` (Illusion Shroud SDK) ~12 modules, `trust_terminal` (TRust / Smart Terminal) ~20 modules, the canonical kernel (`jayce_kernel_src`, ~80 engines), `astraxis` (mission presets + PathSense), `apodeixis` (the provable language), `cosmic_map` (fleet UI), and `master_console` (brain-swarm replication).

Three release personas (see §13): standalone sidecar daemon (Release 1), Jayce Security SDK (Release 2), and bare-metal Observatory OS on the Jayce kernel (Release 3).

2. Core Hardening

2.1 Binary / build-level anti-analysis (`Cargo.toml`, `.cargo/config.toml`)

- **Static musl build only** — miniguard and miniguard-sentinel link statically against musl; **no glibc/libc .so dependency surface** for a reverse engineer to attach to.
- **Symbols stripped, LTO on, codegen-units=1** — release profile ships **no function names** in the binaries and blurs the call graph.
- **Capability-revealing strings obfuscated** (`obfstr 0.4`, in `seal.rs`) — `seal-manifest` path, socket paths, sentinel heartbeat/freeze-log paths, binary paths exist only as decrypted-at-runtime buffers; `strings` finds nothing. On-disk-discoverable paths (policy lists) stay plain — documented residual.
- **argv/envIRON scrub** (`scrub_argv_envIRON`) — `/proc/self/{cmdline,envIRON}` are empty after startup.

2.2 Runtime self-protection (`self_protection.rs`, `seal.rs`)

- **PR_SET_DUMPABLE=0** — disables ptrace/core-dump of the daemon.
- **Tracing is lethal** — a foreign TracerPid → self-SIGKILL → clean systemd restart (with escalation tallies).
- **PDEATHSIG** wired so disposable components die with their parent.
- **analysis_detector.rs** — watches for reverse-engineering activity *targeting the daemon*: debuggers (gdb/lldb), tracers (strace/ltrace), dynamic instrumentation (frida), disassemblers (radare2/IDA/Ghidra/Binary Ninja), and any process opening `/proc/<daemon_pid>/mem|maps` (root bypasses DUMPABLE=0 — this catches it). Targeted analysis/memory reads escalate to attack memory + illusion layer + paranoid mode.
- **Antifreeze sentinel, separate process** (`miniguard/src/bin/sentinel.rs`, `sentinel.rs`): SIGSTOP-ing the daemon freezes anything *inside* it, so an independent ~1.3 MB static binary polls `/proc/<daemon>/status` every 2 s (State: T → SIGCONT + FREEZE_EVENT log), with an escalation ladder (≥3 freezes in 60 s → SIGKILL clean restart), culprit capture, and mutual watch (daemon restarts a dead sentinel).
- **self_identity.rs** — the ONE authoritative “is this the daemon itself?” answer, from pid + exe + service cgroup root (`/proc/self/cgroup`), never by name (the daemon masks its name). Self-activity is routed to an integrity path and **never self-enforced** (prevents a self-kill restart loop).

2.3 Filesystem / namespace sandbox of the daemon

- **Landlock per-thread sandbox** (`landlock_sandbox.rs`) — `landlock_restrict_self` binds only the calling thread + threads it creates;

the tokio runtime is built by hand with `on_thread_start(|| apply_default_sandbox_quiet())` **after** the sandbox is in place, so every worker is sandboxed at birth with **no pre-sandbox window** (a prior `#[tokio::main]` ordering once left the mgmt socket escape open — that ordering is the regression-guarded canon).

- **namespace_sandbox.rs / process_sandbox.rs / mg_sandbox.rs** — process isolation surfaces (namespaces, seccomp, supervisor) for governed children.
- **seccomp_filter.rs** — syscall allow-list for sandboxed children.
- **Observability is socket-only** (`seal_logger.rs ring + logs command`) — `journalctl` shows nothing from the daemon.

2.4 Immutable deployment (`seal.rs`, `scripts/seal.sh`)

- **chattr +i** on both binaries, `/etc/miniguard/seal.hashes`, systemd units, config.
- **Read-only bind-mount of `/usr/local/bin`** (`--mount-ro`) as a second tamper layer.
- **Runtime re-verification every 120 s** (daemon tick) / 60 polls (sentinel): hash drift → daemon escalates to paranoid + audit; sentinel **SIGKILLS a drifted daemon** (one-shot, then ALARM — no kill loop).
- **Kernel lockdown (integrity)** enabled if `/sys/kernel/security/lockdown` present; Secure Boot / TPM surfaced by `seal` / TUI “Seal” line.

2.5 System-level hardening knobs

- **sysctl.rs** — kernel-sysctl hardening; aggressive set in paranoid mode.
- **host_integrity.rs** — host-integrity spot checks.
- **iptables_monitor.rs / loopback_monitor.rs** — watch for netfilter/loopback tampering.
- **bpf_pin_integrity.rs** — verifies integrity of pinned eBPF maps.
- **mem_protection.rs** — memory-protection anomaly checks.
- **no_ambient_trust.rs** — removes/refuses ambient trust defaults (fail-closed posture).

3. Detection & Monitoring

The daemon maintains a **state-driven scanner director** (`main.rs`, `ScanDirector`) that scales scan cadence with threat state: every tick when Hostile/Paranoid, every 2nd tick otherwise, and skips the scan entirely when Calm + unchanged fingerprint (`/proc` PID set + `/tmp`, `/var/tmp`, `/dev/shm` mtimes). Heavy scans run off-thread into batch snapshots; enforcement latency stays ≤ 1 tick.

All detections fold into EventSource-tagged BrainEvents (brain.rs) and escalate through the Guardian Brain (§4). Event sources include: NetworkMonitor, EbpfEnforcer, ProcessDetector, PersistenceDetector, RootkitDetector, HostIntegrity, SelfProtection, FileWatcher, AnomalyDetector, ExtensionMonitor, ClipboardGuard, CredentialGuard, WalletGuard, FormjackingDetector, DownloadGuard, ServiceWorkerGuard, RedirectGuard, SessionGuard, MemProtection, InputGuard, BrowserStorageMonitor, RootedMalware, WebsocketGuard, ConnectionGuard.

3.1 Process-level detection (guardian_proc.rs)

- Executable-level ThreatDetector scanning binaries/files for malicious patterns; chain-depth analysis (DEEP_CHAIN_THRESHOLD) spotting deep process bridges; browser-guard scans; web-dir + tmp-script scanners; ring-buffered alert history (ThreatAlert with severity levels).
- **process_profiler.rs** — per-process behavior profiling.
- **behavior_policy.rs** — declarative behavior rules with deny/allow effects.
- **no_ambient_trust.rs** — treats unseen binaries as untrusted by default.

3.2 Persistent-process / root-level malicious activity

- **rooted_malware.rs** — rooted-malware evaluator combining kernel verdict → ancestry → baseline trust; CRITICAL_INFRA_EXES (sshd/nginx/postgres...) confirm on 2 tier-0 signals instead of 3.
- **rootkit_detector.rs** — rootkit hooks/detection heuristics.
- **persistence.rs / persistence_harden.rs / autorun_detector.rs** — persistence-author mechanism detection (systemd/cron/autostart) + hardening.
- **container_monitor.rs** — scans /proc every 30 s for the container threat surface: runtime detection (dockerd/containerd/crio/kubelet/podman), cgroup inventory, docker.sock mounts, CAP_SYS_ADMIN privileged containers, host namespace sharing, sensitive host mounts, and escape paths (/proc/1/mem, release_agent, /etc/shadow).
- **sandbox_escape.rs** — sandbox-escape attempt detection/heuristics.
- **fileless_injection_detectors** — memfd_monitor.rs, memory_scanner.rs, code_sniffer.rs, wasm_detector.rs, process_mask.rs (masked/fileless binaries).

3.3 Privilege / capability layer

- **capability_broker.rs** — capability granting/denial broker.
- **capability.rs (trust_core)** — capability token model.
- **no_ambient_trust.rs** — refuses ambient capability assumptions.

- **automata.rs** — automata-modeled behavior monitoring.

3.4 Network detection (**guardian_net.rs**, **network_monitor.rs**)

- Host-wide HostMonitor producing ThreatLevel + IP/beacon lists; exfiltration rate tracking; beacon-IP detection; gateway/interface monitoring.
- **dns_rebind_guard.rs / redirect_guard.rs** — DNS-rebinding and host-header redirect protections.
- **transparent_proxy.rs** — transparent-proxy analysis of traffic.
- **loopback_monitor.rs / iptables_monitor.rs** — local radio and firewall tamper detection.
- **tls_config.rs** — enforced TLS configuration posture.

3.5 Browser & web-threat surface

- **formjacking_detector.rs** — form-scraping / payment-skimmer detection.
- **cryptojack_detector.rs** — in-browser coin-mining detection.
- **download_guard.rs** — suspicious-download vetoing.
- **service_worker_guard.rs / websocket_guard.rs / webusb_guard.rs / dns_rebind_guard.rs** — web-API abuse surfaces; USB_SAFE_PREFIXES allowlist for legitimately-HID-holding whitelisted daemons.
- **extension_monitor.rs** — browser-extension monitoring.
- **browser_storage_monitor.rs** — browser storage (localStorage/IndexedDB) inspection for planted malware.
- **wallet_guard.rs** — crypto-wallet access & clipboard-swap protection.
- **frame_guard.rs / image_scanner.rs / document_scanner.rs** — hostile frame/ad-injection detection and image/document content scanning.

3.6 Human-interface / device peripherals

- **input_guard.rs** — keylogger/UHID poisoning detection over evdev.
- **clipboard_guard.rs** — clipboard-swatch and content inspection (incl. a .poison-test.md validation path).
- **camera_guard.rs / mic_guard.rs** — webcam/microphone access detection.
- **guardian_usb.rs / webusb_guard.rs** — USB device & WebUSB policy.
- **credential_guard.rs** — credential-stuffing / password-guard.
- **session_guard.rs** — terminal-session / tty hardening.
- **keyring.rs** — a guarded keyring for secrets.

3.7 Syscall / kernel / integrity signals

- **audit_chain.rs** — hash-chained (HMAC) audit log of every verdict — tamper-evident ordered history.
- **event_correlator.rs / correlation.rs** — multi-signal correlation.

- **anomaly_detector.rs** — behavioral anomaly scoring.
 - **host_integrity.rs / measurement_log.rs / liability_tracker** — integrity checks + measurement-affordance logs.
 - **admission.rs** — admission control gating actions.
 - **no_ambient_trust.rs** — fail-closed posture baseline.
-

4. Enforcement & Response

4.1 Guardian Brain & escalation (brain.rs)

- **Four-state threat ladder** (Calm → Suspicious → Hostile → Paranoid) driven by EventSource-tagged events with per-source escalation thresholds; adaptive thresholds that tighten as the system sustains pressure.
- **Autonomous escalation** synthesizes multi-stage correlations into decisions (Isolate / Restrict / Kill / EnableParanoidRestrictions).
- **wants_enforcement()** gates eBPF enforcement (Hostile/Paranoid by default; `enforce_in_suspicious` is an opt-in knob).
- **force_paranoid(reason) / set_judge_distrust** — fail-closed lockdowns.
- **Learning:** `tick_with_kernel()` folds a kernel snapshot into verdicts; `record_enforcement_outcome / record_false_positive` drive self-correction.
- Memory graph (MemoryGraph → `attack_memory.rs/memory_graph.rs`) persists learned patterns (HMAC-keyed, integrity-verified) across restarts.

4.2 Enforcement primitives

- **enforcement.rs** — `try_kill / try_quarantine`, each re-checking ancestry before acting and **refusing self-PIDs**.
- **guardian_adapter.rs** — constraint-adapter interface (used by Apodeixis RCF missions; **deterministic, no I/O**).
- **admission.rs** — admission control pre-gate.
- **incident_response.rs** — declarative incident-response actions.
- **recovery.rs** — recovery procedures after incidents.

4.3 Network enforcement (guardian_fs.rs firewall, ebpf_enforcer.rs)

- **iptables firewalling:** JAYCE_INPUT / JAYCE_OUTPUT chains with dynamic per-IP DROPs, per-destination DNS rate-limit, egress filtering; **always- cleanup** (`remove_all`) wired to graceful stop AND crash so no half-configured chain is ever left behind.

- **eBPF enforcer**: packet drop maps, exec-block maps (fnv1a hashes → drop), IP blocklist, self-defense; enforcement only in Hostile/Paranoid (± Suspicious opt-in). IPv6 support fixed in a later audit.
- **connection_guard.rs** — Apodeixis-verified connection integrity (§10).
- **fast_path.rs** — trusted-IP allowlist on dynamic blocks (avoids collateral).

4.4 Filesystem enforcement (**guardian_fs.rs**)

- **FileGuard / DnsFilter / FirewallManager / ProcessGuard** — filesystem quarantine, DNS-category filter, firewall manager, process guard.
- **mg_file_monitor.rs** — continuous file monitor for governed processes.

4.5 Process / namespace enforcement

- **process_sandbox.rs / namespace_sandbox.rs / process_mask.rs** — sandbox/namespace/masking for targets.
- **mg_sandbox.rs** — CliSandbox used by the mg-cli governor.

5. Deception: Active Defense

This subsystem actively **misleads** an attacker rather than only denying. It spans the user-mode daemon, the kernel, and the shroud.

5.1 Illusion Bubble (**miniguard/src/illusion_bubble.rs**)

A command gate that, for a dangerous command, does not merely reject — it can Allow, Reject, or **Deceive { response }**. When deceiving it: - Records the MITRE **ATT&CK technique** (a full enum over all 14 Enterprise tactics, 100+ techniques) for intelligence (TechniqueObservation ring). - Returns a **fake but plausible response** (fake_responses: cat /etc/shadow, id, whoami, uname -a, ifconfig/ip addr, ...). - **Wastes attacker time** with realistic-looking output while **planting honeytoken markers** it can later detect.

Filesystem honeytokens (init_honeytokens): fake-but-look-real decoys with unique markers (MG_HONEYTOKEN_*): a fake /etc/shadow, a fake /etc/passwd with a “honeyuser”, a fake SSH private key (/tmp/.honey_rsa), and a fake credential dump. Surfaces: is_honeytoken_path, get_honeytoken, honeytoken_marker. If an attacker ever returns a marker through any channel, that is a *confirmed* theft signal.

5.2 Kernel-side Dual-Layer Illusion (`jayce_kernel_src/src/kernel/illusion.rs`)

The **Perception Control Membrane** normalizes externally-visible **timing, identities, and coarse outcomes** as probe pressure rises, without changing internal truth: - **Probe scoring** — `record_probe_weight` (light/suspicious/hostile = 1/4/8), suspicion score with per-tick decay, states Calm / Suspicious / Hostile / Quarantined. - **mask_timing** — normalizes observed timings to hide timing side-channels. - **mask_memory_address** — obfuscates real addresses returned to probes. - **generate_fake_code / visible_system_info / external_status / filter_event_for_external** — decoy code, masked system info, status masks, and rewritten/dropped events for external observers. - **timing_masks / address_masks / identity_masks / status_masks / event_drops** counters expose exactly which deception surface is active.

5.3 Shroud counters (`illusion.rs ShroudMetrics / trust_core shroud_*`)

- DDoS blocks + rate-limit trips, firewall drops, peripheral alerts/blocks, wireless blocks + **deauth detection**, prompt-injection blocks + pattern-matched, integrity checks/violations, capability/ring violations — all behind a global `shroud_set_active` kill-switch.

5.4 Paranoid mode — deception-driven lockdown (`paranoid_mode.rs`)

One toggle that simultaneously: expands the illusion-bubble denylist to block all non-essential commands, **empties process exclusions** (monitor everything), lowers exfil thresholds, tightens beacon detection, blocks more egress ports and VPN domains, applies a stricter `sysctl` set, raises audit verbosity, blocks more DNS categories, routes more to the Void Pit, and lowers kernel-enforcement thresholds. Activation reasons: User, HostSuspicion, SelfProtection, Manual, JudgeDistrust. A 10-minute “sustained” gate prevents accidental self-lockouts.

5.5 Attack memory / learning (deception feedback) (`attack_memory.rs`)

A tamper-evident store of `AttackPatterns` (syscall sequence, capabilities, files, network endpoints, verdict, severity) with **edit-distance similarity matching** to recall “seen-before” attack shapes, `matches_known_attack`, threat-score prediction (`predict_and_update`), component host/risk models, and **impossible-state invariants** (`InvariantCheck`: `NeverWithCapability`, `NeverCoexist`, `NeverModified`, `MaxRisk`, `MaxCapabilities`, `Custom`). Backed by `memory_graph.rs`

(JVC-compressed archive, HMAC integrity). This is the learning substrate the self-learning feedback loops (§7) feed.

5.6 Deception in the AI-tool launch path & surfaces

- `mg_cli_governor` runs **illusion bubble** → **Apodeixis mission** → **scorecard** before an AI tool spawns; `BubbleVerdict` (~allow/reject/deceive) is part of the brief (see §9.2).
 - **Deception status surfaces:** `kernel_illusion` terminal command and the TUI “Deception Stack” show Hostile-vs-Calm posture and intercepted MITRE probes.
-

6. Trust Engine & Identity

Before deciding how to treat anything, the system decides *who is asking* — and it keeps the trust decision deterministic and revocable.

6.1 Process-ancestry trust (`trust_origin.rs`)

Walks the PPid chain; trusts processes descended from user-login sessions / system / browser launchers. `TRUSTED_DESKTOP_COMMS` and `TRUSTED_SYSTEM_COMMS` are the curated allow sets (extend here for a new distro/desktop). This is the base layer of `rooted_malware` trust ordering.

6.2 Baseline hash snapshot trust (`baseline_trust.rs`)

After daemon start, hashes every root-owned executable during a 120 s warmup. Afterwards a root binary is trusted **only while its content matches the baseline** — post-warmup arrival, drift, or deletion ⇒ untrusted. Consumed by `rooted_malware.rs` alongside ancestry trust.

6.3 Kernel trust coprocessor client (`trust_client.rs`)

Socket client to the Jayce trust map (`/tmp/jayce-trust.sock`, JSON, circuit-breakered); `verify_trust(pid)` asks the kernel for a verdict, with a local fallback when unreachable (documented, fail-open fallback — the judge link is a *separate* domain: the engine-lineage agreement, §8).

6.4 Rooted-malware evaluator (`rooted_malware.rs`)

Decision order: **kernel verdict** → **ancestry** → **baseline**. Critical infra services confirm on 2 tier-0 signals instead of 3 (`CRITICAL_INFRA_EXES`). `enforcement.rs` / `brain.rs` / `guardian_proc.rs` all consult trust before acting, and skip sandbox/kill commands for trusted PIDs.

6.5 Authoritative self-identity (`self_identity.rs`)

pid + exe + service-cgroup root (`/proc/self/cgroup`); never by process name.
`is_self_pid` is the single source of truth; self is routed to integrity monitoring but **never self-enforced** (§2.2).

6.6 Trust lifecycle & no ambient trust

- **`trust_lifecycle.rs`** — lifecycle of trust grants.
 - **`no_ambient_trust.rs`** — fail-closed default; unseen = untrusted.
 - **`admission.rs`** — admission gate re-verifies trust at ingress.
 - **`trust_harden.rs`** — hardens trust decisions under attack.
 - **`cohost.rs` / `cohost_compat.rs`** — trust integration when the daemon co-hosts with other systems.
-

7. Integrity, Persistence & Recovery

7.1 Shadow backup & self-heal (`shadow_backup.rs`, `file_watcher.rs`)

Keeps baseline shadow copies of security-critical files (`/etc/passwd`, `/etc/shadow`, `/etc/group`, `/etc/gshadow`, `/etc/sudoers`, `/etc/ssh/sshd_config`, `/root/.ssh/authorized_keys`, `/etc/crontab`) under `/var/lib/jayce/miniguard/shadow/` and **auto-restores them on tamper** (content drift or deletion). `/etc/ld.so.preload` is **forbidden-on-sight** (deleted). The attacker's version is preserved in `shadow/evidence/` and its hash feeds attack memory. **Shadow-poison → restore refused**. Admin re-baseline: delete a shadow file — live file becomes the new baseline. `file_watcher::CriticalFileWatcher` adds inotify-style critical-file watching.

7.2 Host-integrity checks (`host_integrity.rs`, `measurement_log.rs`)

Spot-checks on host state; measurement-affordance log captures what was observed for later forensics.

7.3 Nomadic persistence detection (`persistence.rs`)

Tracks persistence-author mechanisms; `persistence_harden.rs` closes them off; `autorun_detector.rs` catches new autostart entries.

7.4 The Void Pit (`jayce_kernel_src/src/kernel/engines/void_pit.rs`)

canonical; ported into `jk::void_pit`): a kernel-side quarantine/quarantine-storage vault for detected violations; `kernel voidpit` terminal command inspects/manages it; `paranoid_mode::void_pit_aggressiveness()` routes more evidence into it under pressure.

7.5 Recovery & incident response

- **recovery.rs** — recovery steps following an incident.
- **incident_response.rs** — declarative IR actions.
- **resilience.rs** — component-resilience behaviors.
- **updater_safe.rs** — update-path safety gating (see §10).

7.6 The self-learning feedback loop

The 4-layer loop **converts every attack into hardening**: 1. Attack memory & threat graph (EWMA scores) → 2. Autonomous escalation (eBPF drop, sandboxing, **deception layer**) → 3. State hardening (thresholds, profiles) → 4. Feedback-driven tuning (adaptive thresholds, baseline rebuilding). Every probe strengthens rather than merely alarms.

8. Deterministic Kernel & Proof-Checked Enforcement

8.1 The Jayce kernel bridge (`miniguard/src/kernel_bridge.rs`, `jayce_kernel/`)

The daemon drives the canonical kernel cell (`jk::`, ported from `jayce_kernel_src/` at build time — edits happen in `jayce_kernel_src/`, never in the port). Engine dispatch is tick-scheduled by `spine::should_engine_tick`. - **Determinism contract** — no entropy, no wall-clock in kernel decisions; engines fire iff $(seq \ \& \ (d-1)) == 0$ or $seq \% d == 0$ (power-of-two budgets). -

Determinism verification — endpoint-invariance hash (FNV-1a32, seed `0x5350_494E "SPIN"`) over compile-time invariants; the daemon can *prove* the kernel has not drifted. - **Truth chain** — $H_i = \text{HMAC}(\text{key}, \text{data}_i \parallel H_{\{i-1\}} \parallel seq_i)$ tamper-evident ordered log of kernel events. - **Defense-in-depth**

structures: `void_pit`, `temporal_proof_ledger`, `doorman` (capability tokens), `constitutional_shadow_layer`, `burn_pit`, `self_healing_causality_mesh`, `temporal_storage_ledger`, `stack_canary`, `impossible_state_detector`, `cold_boot_defense`, `iommu_policy`, `pci_config`,

virtual_device_personalities, etc. (~80 engines total under jayce_kernel_src/src/kernel/engines/).

8.2 Host engine set (jk::engines::host::*)

automata_monitor, burn_pit, cluster, constitutional_shadow, doorman, hardware_info, illusion_layer, impossible_state, predictive_horizon, self_healing, service_supervisor, truth, vitals + network_verdict (re-exported as jk::network) + alien_bridge (the LASV vision ingest point).

8.3 Apodeixis proof-checked missions (apodeixis/, astraxis/missions.rs)

An in-process, deterministic, step-budgeted (100k steps) provable language embedded in the Smart Terminal (term languages). Every eval goes **lex** → **parse** → **typecheck** → **theorem/proof check** → **run**. An Apodeixis mission whose theorem proof fails **never executes** (no side effects). IFC labels @Secret/@Public with runtime flow enforcement and a declassify() escape hatch. (cap is a reserved keyword — never use as a variable name.)

8.4 GuardianAdapter constraints (trust_core/src/guardian.rs)

Deterministic, no-I/O, fail-closed constraint application for RCF missions. Presets: default_robotics_constraints (speed ≤ 2 m/s, per-move displacement ≤ 10 m, force ≤ 50 N, distance-from-origin ≤ 10 m, battery ≥ 15%, collision risk ≤ 0.8, motor thermal delay) for robot/vehicle/drone kinds; default_infra_constraints otherwise. Missing telemetry signal = zero → **fail-closed REJECT**. Only PERMIT verdicts proceed.

8.5 RCF / Robot Command Fabric (astraxis/rcf_compiler.rs, kernel_robot_command_fabric.rs)

Guardian-gated, deterministic actuation: kernel_rcf does a pure constraint check (PERMIT / DELAY / REJECT); kernel_mission runs a proof-checked mission with signal() telemetry injection and emit_action() actuation proposals filtered by the GuardianAdapter. Sealed named presets (robot-e-stop, robot-path-clear, etc.). The kernel also compiles intent-level RCF programs over the serial VM channel (kernel_vm_rcf, §8.6).

8.6 Kernel coprocessor (judge) + engine-lineage agreement

- **kernel_vm.rs** launches the bare-metal Jayce kernel in a lightweight QEMU VM and bridges JAYCE_TLM: serial telemetry to the Cosmic Map pipe. **Fail-**

closed: if QEMU/image missing the daemon continues without the coprocessor (documented `NotApplicable` deployment choice).

- **agreement.rs** — every tick a proof-checked Apodeixis mission confirms the judge is alive (telemetry within 10 s) **and** its engine lineage (SHA-256 over the 18 synced engine sources, `ENGINE_LINEAGE`) matches the daemon's. Any failure/disagreement/tamper → `MissionFailed` → **distrust, never trust**; judge absent/killed or lineage mismatch → paranoid latch (`ActivationReason::JudgeDistrust`), cleared only after 3 consecutive aligned ticks. Verdict gauge `judge_agreement`: 0 = no coprocessor, 1 = trust, 2 = latched distrust.
- **Dual model (documented):** without a coprocessor the daemon is sole authority; with one, the kernel is the *judge* and the daemon is the *enforcer* — with a decision **never goes to the kernel's privileged world** (the design keeps the host separate).

8.7 Real-kernel serial command channel (`kernel vm rcf|raw|ack`)

QEMU stdin → guest COM1 RX → the kernel's Trusted Operator Terminal reads bounded (80-char, printable) lines. `rcf <id> <budget> <op> [args...]` submits intent-level RCF programs; every submission acked on serial as `JAYCE_RCF: id=N ok=B [err=parse]`. Callers use the auth-socket `vmcmd:<line>` (validated by `KernelVm::validate_line`) or `TRust kernel vm` subcommands.

9. AI Containment & AI-Content Safety

This is MiniGuardian's AI-specific defense. The core principle of the containment essay: **the AI is not the trust boundary — the container around it is**. "A local model can be tricked into *retrieving* a poisoned file, *pasting* it into its own context, and *behaving* according to that file's instructions. The threat isn't in the weights; it's in the pipeline that feeds the weights." Containment therefore sits in the pipeline, as stacked walls.

9.1 The five memory-poisoning defenses (`trust_core/src/memory_poison_defense.rs`)

Detection-only, deterministic, designed to catch what string-pattern prompt guards cannot: 1. **RAG Corpus Guard** — document fingerprints; detects adversarial embedding drift; scores retrieval-frequency anomalies. 2. **Context Window Guard** — monitors system-prompt displacement; detects when defensive context is pushed out by token flooding. 3. **Conversation Summary Validator** — detects injection payloads hidden inside summaries / compressed context windows. 4. **Retrieval-Time Injection Guard** — scans retrieved docs for adversarial triggers that only activate when retrieved. 5. **Cross-Session Persistence Monitor** — tracks what

persists across session boundaries; detects unauthorized long-term-memory writes. Verdicts classify into `MemoryThreat` / `MemoryAction` and feed `RuntimeStatus.ai_mem_*`.

9.2 The AI-governed tool pipeline (`mg_cli_governor.rs` + friends)

`mg-cli` is a **launch-time wrapper** governing one binary you explicitly launch through it — and it is the only place full **filesystem pinning** (Landlock) happens today: 1. **Auto-Detect** the AI tool (`mg_ai_detector.rs`, `ai_adapters.rs`) from binary signatures, process trees, env fingerprints, API flags (`AIToolCategory`: `cli` / `editor` / `agent` / `custom`; `KNOWN_AI_EDITORS`, `KNOWN_AI_AGENTS`, `AI_ENV_FINGERPRINTS`). 2. **Landlock shell** (`mg_sandbox.rs`) + workspace scoping; seeds the user's existing tool config so sessions/state survive while future writes stay contained. 3. **Illusion Bubble → Apodeixis mission → scorecard** gate (`illusion_bubble::BubbleVerdict`, `mg_apodeixis.rs`, `mg_scorecard.rs`), reported via `cli_gov_telemetry`: frames (`mg_ai_bridge.rs`). 4. **Continuous file monitor** (`mg_file_monitor.rs`) + rule engine (`mg_rule_engine.rs`, `CliRuleEngine/RuleEffect`) + dynamic risk score (`CliScorecard`). A governed tool **must reach its own model endpoints** (R1) — the global DNS cap that once broke opencode's remote model is default-OFF; its per-tag egress replacement is gated behind the AIE risk gate (§9.5).

9.3 The AI content-evaluation socket (`eval_server.rs`)

A **separate, tokenless, read-only** endpoint at `/var/lib/jayce/miniguard/eval.sock` (0666, in the root-owned 0755 data dir so a governed AI can connect but not replace the socket or read the 0600 token). It exposes ONLY pure evaluations: - `prompt_eval:<base64>` — PromptGuard verdict, - `mem_scan:retrieval|summary:<base64>` — MemoryPoisonDefense scans, - `mem_context:<total>:<system>:<external>` — context-window drift check, - ping. Used by the governed opencode plugin `miniguard-firewall.js` to route tool results / compaction summaries / context composition through the deterministic scanners **before** they reach the model's context. Fail-open (logged) when the daemon is down; the hard boundary stays Landlock.

9.4 Three-layer PromptGuard (`trust_core/src/shroud_prompt_guard.rs`)

A weighted, multi-layer prompt-injection guard: - **layer 1** pattern match → **layer 2** intent/behavior heuristic → **layer 3** pluggable semantic analyzer (`SemanticAnalyzer`). Produces `PromptVerdict`, per-class `safe_alternative`, `related_cap_bits`, `is_never_allow`, escalation threshold, system-prompt fingerprinting, output filtering (`filter_output`), `ConversationContext` tracking,

event log, rate limiting, and stats. cap bits tie denials to capability tokens. Exposed via `shroud_bridge.rs` and the `eval_server`.

9.5 The AIE (AI Workspace Envelope) data layer — Phase 0 detection-only

The future-proof adapter/registry model, shipped **detection-only** and gated: - **ai_adapters.rs** — pure-data adapter registry (identity signals, session-state templates, model endpoints per tool). One new AI tool = one adapter entry; detection code never changes. - **ai_envelope.rs** — in-memory **tag map** of observed AI processes (EnvelopeTag: pid, tool/adapter, kind, confidence, model endpoints, fs_pinned, first/last seen), bounded + age-pruned. **Contract: no cgroup writes, no network/DNS enforcement.** fs_pinned: true only for sessions launched through `mg-cli`. - **Surfaced** via `RuntimeStatus.ai_containment_tags / ai_containment_tag_count / ai_envelope_ready`, and the `miniguard-status` AI Containment panel. - **Deferred (hard gate):** cgroup tag move, per-tag egress allowlist, GUI exec interception, registry session-state engine. Any egress rule that is host-wide or mis-scoped re-creates the `opencode` infinite-retry loop (R1 violation). `docs/AIE_IMPLEMENTATION_RISK.md` lists the catastrophic-failure matrix and the kill-switch / dry-run / chaos-suite preconditions that must pass first.

9.6 AI aggregation in the daemon

- **mg_ai_detector.rs / mg_ai_bridge.rs** — detection + telemetry frames (`KernelAiBridgeClient, CliGovTelemetryFrame`).
- **mg_apodeixis.rs** — `evaluate_action` gates launch/file actions by a proof-checked Apodeixis mission context.
- **mg_scorecard.rs** — risk score + `ActionDecision` for governed launches.
- **session_guard.rs / process_profiler.rs + the passive /proc scan** feed the envelope every 6 ticks.
- **self_identity + integrity** on self-activity; **memory_graph** keeps learned patterns for the AI-safety pages (TUI a key in `miniguard-status`).

9.7 Offline, no-telemetry posture (trust architecture)

The substrate is deliberately **architecturally offline** — no cloud, no behavioral fingerprint upload, no dashboard in someone else's datacenter. The eval socket, log ring, sentinel, and kernel coprocessor are all local. "The trust boundary is only trustworthy if it doesn't answer to anyone but you." (The Master Brain swarm model remains an opt-in roadmap item, §11.)

10. Communications, Updates & Sidecars

10.1 Management socket (`socket_server.rs`, `tls_config.rs`)

Unix-domain management socket, **root-only 0600 auth token required as the first line of every command**, with per-peer auth-failure rate-limiting and a large command surface (`seal`, `logs`, `status`, `forensic`, `update ...`, `term:`, `kernel ...`, `vmcmd:`, ...). Auth-failure stats (`auth_failure_stats`) fold into the connection-integrity logic.

10.2 Gossip/peer transport (`gossip_transport.rs`)

TLS-mandatory, mutual-auth, HMAC-SHA256, chain-hashed, per-peer rate-limited peer messaging. Peer state tracked with a 10-min TTL for the Cosmic Map roles.

10.3 Apodeixis-verified connection integrity (`connection_guard.rs`)

Every tick the daemon folds raw security facts (GossipSecurity: auth/HMAC/TLS-handshake/rate-limit/version counters + quarantine flag; `socket_auth_failure_stats`) into an embedded, proof-checked Apodeixis mission whose expected (`action`, `target`) must match the Rust-computed verdict exactly. - Any mismatch → `MissionFailed` → worst-case action, never a downgrade. - **Fail-closed**: gossip violation ($\geq \text{SOCKET_AUTH_STORM}=5$ auth failures, or HMAC/TLS-handshake/version failures) → transport quarantined (input dropped, live links torn down, outbound refused) + brain escalation (threat 60, severity 2); socket violation → brain escalation only (threat 61). - Single violation latches immediately; clears only after 3 consecutive healthy ticks. Verdict gauge `connection_integrity` (0 = healthy, 1 = latched). - Trust-sidecar absence is NOT a violation (documented fallback).

10.4 Signed OTA updates (`update_manager.rs`, `updater_safe.rs`)

Governed by an embedded proof-checked Apodeixis safe-update mission that vetoes installation unless the host is calm, the deployment is sealed, and the manifest signature + binary hashes are valid: - Manifest = signed wrapper (manifest + signature over Ed25519), per-binary sha256 + size. - Flow: verify manifest → stage to `/var/lib/jayce/miniguard/updates/<version>/` → run safe-update mission (fail-closed on any doubt) → stop daemon + sentinel, remount `/usr/local/bin` rw, rotate backups, swap, re-apply `chattr +i` + read-only mount → restart + health-gate → auto-rollback to `.bak.1` if the gate fails. - Modes: `auto_check`, `auto_apply` (default off), `allow_rollback`.

10.5 Sidecar protocol & bridges

- **proxy.rs** — SidecarRequest/SidecarResponse wire protocol behind the same auth gate for command forwarding.
 - **trust_terminal_bridge.rs** — term:<line> → SmartTerminal; every line hits the **Prompt Guard first** (fail-closed); shell pipelines, inline code, and kernel commands flow through here.
 - **shroud_bridge.rs** — daemon↔shroud integration point.
 - **cohost.rs/cohost_compat.rs** — co-hosting integration.
-

11. Fleet Visibility, Forensics & Observability

11.1 Cosmic Map fleet view (cosmic_map/)

A single packaged binary. **Daemon** (root) emits JAYCE_TLM: telemetry frames to /tmp/jayce-telem.out every tick (cosmic_bridge.rs): this host (cluster node 1) + every gossip peer, threat-level → status mapping (Calm=online, Suspicious=degraded, Hostile/Quarantined=offline-red). **UI** (miniguard --map) reads it with **no root**. Two frontends over one headless core (state + telemetry_reader): the egui Observatory window and the ratatui terminal SaddleMap (testable headlessly). Every frame also carries the Robotics Saddle surface (per-unit keys + saddle scalars), snapshotted inside the kernel lock (lock order always jk → runtime).

11.2 Forensic report (forensic_report.rs)

Full incident-review dump over the auth socket (forensic): audit chain + kernel truth chain + void-pit roster + **attack memory + playbook + deception observations** + artifact inventory + seal posture.

11.3 Audit chain & SIEM (audit_chain.rs, siem_export.rs)

HMAC hash-chained audit log of every verdict (GuardSource + Verdict) and an export surface (siem_export) for external collectors.

11.4 Observability (socket-only) & health

- **seal_logger.rs** ring (logs, last 400) via socket — journalctl silent.
- **health.rs (HealthServer)** — health endpoint (daemon/socket/heartbeat).
- **runtime_status.rs** — RuntimeStatus shared state
incl. ai_containment_*, ai_mem_*, connection_integrity, judge_agreement.

- **json_logger.rs / measurement_log.rs / canary.rs** — structured logging / measurement affordances / injection canaries.
 - **metrics.rs / latency_tracker.rs / tick_scheduler.rs / tick_phase.rs / tick_budget.rs** — timing/health of the tick engine.
 - **miniguard-status** TUI + miniguard-saddle console (§12). Master Brain / swarm immunity = opt-in roadmap item.
-

12. Robotics Saddle & Actuation Gating

12.1 The saddle personality (robotics_saddle.rs)

`miniguard --saddle(or [robotics_saddle] enabled = true)` switches the daemon into the robotics-OS personality: host-desktop noise subsystems are dropped (gossip, transparent proxy, dev mode, host monitor, persistence detector, kernel bridge, threat detector) and the mandatory robotics invariants are pinned: kernel coprocessor judge, rooted-malware evaluator, security enforcer, **illusion-bubble command gate**, and **hash-chained audit log**. `assert_invariants` runs at startup and is **FATAL** on any waived invariant (`robotics_saddle.enforce_host_guards = false` is the only signed waiver, for bench rigs).

12.2 The runtime (trust_terminal/src/robotics.rs)

`kernel robot|drone|vehicle|swarm|saddle` terminal commands are backed by a real process-global `RoboticsRuntime` (shared by the REPL and the daemon socket bridge) — not hardcoded strings. Every actuation is a fixed **three-stage gate**, all deterministic and fail-closed: 1. **GuardianAdapter** with `default_robotics_constraints` ($\text{speed} \leq 2 \text{ m/s}$, $\text{per-move displacement} \leq 10 \text{ m}$, $\text{force} \leq 50 \text{ N}$, $\text{target distance-from-origin} \leq 10 \text{ m}$, $\text{battery} \geq 15\%$, $\text{collision risk} \leq 0.8$, motor thermal delay). 2. **PathSense** spatial clearance from the unit's sensor spectrum — no data is DENY; a near-miss latches the **system-wide e-stop**. 3. **Astraxis Controller** `move_dir` per axis — each step vetoed by its own Apodeixis mission and re-checked against the safety ground truth (`astraxis/controller.rs`, `astraxis/safety.rs`, `astraxis/sensor_forge.rs`, `astraxis/path_sense.rs`). Every outcome lands in a bounded (256) actuation ledger (`kernel saddle / kernel saddle tpl`); `kernel saddle estop|release` drive the system-wide latch. Tests touching the runtime **MUST** hold `robotics::lock_and_reset()`.

12.3 LASV vision link

`robot vision <id> <obstacle_m> [targets] [risk] [frame]` — ingests a Live-Action-Sensor-View frame: (1) obstacle distance recorded into the kernel's alien

bridge (§27, `jk::engines::host::alien_bridge::record_sample`, normalized mm amplitude `0..=65535`), (2) SensorForge mission-verified ingest (taint-aware; a veto rejects the frame), (3) unit's live `sensor_distance` + stored `VisionFrame` update, so the next move/grasp gate runs `PathSense` on vision-derived clearance and the guardian on fused risk (`max(distance_risk, vision_risk)`). **CRITICAL:** `jk::` calls here are made directly — the daemon already holds the kernel lock; re-acquiring deadlocks. Move-gate ordering is `PathSense` first, then guardian at the recommended speed.

12.4 Operations console

`miniguard --tui / binary miniguard-saddle: units table, ratatui SaddleMap, actuation ledger, kernel/judge TPL panes, command bar sending term: lines over the auth socket (token first), telemetry from the pipe (no root for display); --mini single-column monochrome; --touch /dev/input/eventN evdev gesture bridge (tap=send, swipe up/down=scroll, left/right=map fullscreen, long-press=clear; in-app translation — never TIOCSTI).`

13. Deployment Modes, Profiles & Hardening Scripts

- **Three install modes:** `sidecar` (Release 1 daemon on an existing OS), `os-migration / cohost`, and **bare-metal saddle** (Release 3, bootable Jayce kernel). `scripts/deploy.sh install --mode=... --workload=... --posture=...` is the unified installer with an interactive wizard (mode → workload → posture → cluster/chatter) and **five canonical profiles** under `profiles/<workload>-<posture>.toml`.
- **scripts/seal.sh** — sealed production deploy (§2.4): stops services, copies static binaries, writes `/etc/miniguard/seal.hashes`, `chattr +i`, tightens the data dir, enables kernel lockdown, restarts; `--mount-ro` adds the read-only bind mount.
- **scripts/attack_sim.sh** — live attack simulation against the running daemon (reads from the socket log ring; needs `sudo + socat`).
- **scripts/build_kernel.sh** — builds the bare-metal kernel from `jayce_kernel_src/`.
- **scripts/illegal_translation.sh / selinux helpers** — distro-integration (AppArmor/SELinux/seccomp profiles) and policy installers.
- **scripts/deploy.sh** also produces `miniguard-saddle`, `miniguard-status`, and bridges; `docker-compose.yml` + `Dockerfile` mount the containerized presence (note the threat-model caveats in `docs/security_assessment.md`: `network_mode: host`, `CAP_NET_ADMIN/CAP_BPF`, `docker.sock` mounted ⇒ a container daemon compromise ≈ host root — the sealed bare-metal/native path is the hardened deployment).

- **Profile / posture selection** (docs/security_assessment §7.5): user-mode vs critical-systems maps to `security_enforcer.enforce_in_suspicious`, `exclusions.monitor_only`, sandbox-failure behavior (warn+continue vs abort), and `sysctl` hardening.
 - **Config under `/etc/miniguard/`** is root-only and sealed; `config.rs` parses `SidecarConfig` (incl. `[updates]`, `[kernel_vm]`, `[robotics_saddle]`).
-

14. Cross-Cutting Engineering Contracts

These are the invariants the repo treats as **non-negotiable** (the “security contract” in AGENTS.md). Any feature that trades one away is treated as a regression: - **Static musl build only** — no dynamic .so surface. - **Zero warnings** workspace-wide; the full test matrix stays green. - **Capability-revealing strings go through obfstr**; verified with `strings` after any change. - **Observability stays socket-only** — never add journald logging. - **Preserve hardening invariants**: `chattr +i`, `argv/envIRON scrub`, `PR_SET_DUMPABLE=0`, `tracer→SIGKILL`, antifreeze sentinel, seal-hash cross-verification (drift → paranoid / SIGKILL). - **Kernel lock discipline**: no `jk::state` from a daemon thread without `with_kernel_lock()` (not reentrant); robotics lock order always `jk → runtime`. - **Guardian logic deterministic** — no I/O, no randomness, no unbounded loops; missing signals = fail-closed reject. - **New config under `/etc/miniguard/`** is sealed and root-only. - **No long-running external processes in the daemon** — new logic goes to `jk::engines` or sealed sidecar binaries. - **TRust terminal stays fail-closed** (Prompt Guard); never bypass the guard for `term: input`. - **The AIE gate**: anything touching `cgroups` / `per-tag egress` / `GUI exec` / `state engines` ships behind a kill-switch, defaults off, dry-runs first, and only after the chaos suite proves R1–R6. - **Apodeixis proof-domain honesty**: in bare-metal saddle the proof-checker’s known unsound edges (Assume as arbitrary axiom, symbolic-auto Neq/Eq) mean mission guarantees are advisory until the checker is made sound (docs/security_assessment.md §7.4 records the fix list).

15. Appendix: Module → Feature Map

Daemon (`miniguard/src/`) — hardening: `seal.rs`, `self_protection.rs`, `landlock_sandbox.rs`, `namespace_sandbox.rs`, `process_sandbox.rs`, `seccomp_filter.rs`, `self_identity.rs`, `analysis_detector.rs`, `sentinel.rs` (bin), `seal_logger.rs`, `bpf_pin_integrity.rs`, `sysctl.rs`, `host_integrity.rs`, `iptables_monitor.rs`, `loopback_monitor.rs`, `no_ambient_trust.rs`. **Detection**: `guardian_proc.rs`, `guardian_net.rs`, `network_monitor.rs`, `rootkit_detector.rs`, `rooted_malware.rs`, `persistence.rs`, `persistence_harden.rs`, `autorun_detector.rs`, `container_monitor.rs`, `sandbox_escape.rs`, `memfd_monitor.rs`,

memory_scanner.rs, code_sniffer.rs, wasm_detector.rs,
process_mask.rs, capability_broker.rs, automata.rs,
anomaly_detector.rs, event_correlator.rs, correlation.rs,
behavior_policy.rs, process_profiler.rs, credential_guard.rs,
wallet_guard.rs, formjacking_detector.rs, cryptojack_detector.rs,
download_guard.rs, service_worker_guard.rs, redirect_guard.rs,
session_guard.rs, input_guard.rs, clipboard_guard.rs,
camera_guard.rs, mic_guard.rs, guardian_usb.rs, webusb_guard.rs,
websocket_guard.rs, dns_rebind_guard.rs, extension_monitor.rs,
browser_storage_monitor.rs, frame_guard.rs, image_scanner.rs,
document_scanner.rs, ransomware_monitor.rs, mem_protection.rs.
Enforcement: brain.rs, paranoid_mode.rs, enforcement.rs,
guardian_adapter.rs, guardian_fs.rs, ebpf_enforcer.rs,
admission.rs, incident_response.rs, recovery.rs, fast_path.rs,
invocation/limits. **Deception:** illusion_bubble.rs, attack_memory.rs,
memory_graph.rs, paranoid_mode.rs (see §5). **Trust:** trust_origin.rs,
baseline_trust.rs, trust_client.rs, rooted_malware.rs,
trust_lifecycle.rs, trust_harden.rs, admission.rs,
self_identity.rs, exclusions.rs. **Integrity/recovery:** shadow_backup.rs,
file_watcher.rs, host_integrity.rs, measurement_log.rs, sbom.rs,
keyring.rs, licensing.rs, audit_chain.rs, canary.rs,
resilience.rs. **Kernel/proof:** kernel_bridge.rs, kernel_vm.rs,
agreement.rs, alien_bridge.rs, piggyback_codec.rs (+ jayce_kernel/
jayce_kernel_src/, apodeixis/, astraxis/, trust_core/guardian.rs). **AI**
containment: mg_cli_governor.rs, mg_ai_detector.rs,
mg_ai_bridge.rs, mg_apodeixis.rs, mg_sandbox.rs,
mg_rule_engine.rs, mg_scorecard.rs, mg_file_monitor.rs,
ai_envelope.rs, ai_adapters.rs, eval_server.rs, mg_cli.governed
profile, shroud_bridge.rs (+ trust_core/shroud_prompt_guard.rs,
trust_core/memory_poison_defense.rs, trust_core/
semantic_analyzer.rs). **Comms/updates:** socket_server.rs,
gossip_transport.rs, connection_guard.rs, update_manager.rs,
updater_safe.rs, proxy.rs, trust_terminal_bridge.rs,
tls_config.rs, siem_export.rs, json_logger.rs. **Fleet/ops:**
cosmic_bridge.rs, telemetry_reader.rs, forensic_report.rs,
runtime_status.rs, health.rs, metrics.rs, tick_scheduler.rs,
tick_phase.rs, tick_budget.rs, latency_tracker.rs (+ cosmic_map/
miniguard-status/, master_console/). **Shroud (trust_core):** shroud.rs,
shroud_prompt_guard.rs, memory_poison_defense.rs,
shroud_firewall.rs, shroud_network.rs, shroud_peripheral.rs,
shroud_wireless.rs, shroud_hardening.rs, shroud_crypto.rs,
shroud_service.rs, shroud_adapter.rs, capability.rs,
semantic_analyzer.rs, guardian.rs.

Part II — Deep-Dive Subsystems

This part goes one level deeper than §1–§15: the *operating machinery* of the system — the learning loop, the memory substrate, the adapter/bridge/shim seams, the clustering layer, the full shroud, the terminal’s cognition, and the watchdog/sentinel/gate “survivability” surfaces.

16. The Learning System & Cognition

The system “learns” in four interlocking ways — all deterministic and tamper-evident, none uses a neural network:

16.1 Attack-memory similarity learning (`attack_memory.rs`)

AttackPattern records (syscall sequence, capability set, files, endpoints, verdict, severity, source component) are stored and **recalled by edit-distance similarity**: `find_similar/matches_known_attack` recognizes “seen-before” attack shapes, and `threat_score_for_sequence` assigns a learned score. The store is HMAC-keyed and integrity-verified, so an attacker cannot poison history to launder a repeated technique. `PredictionResult::predict_and_update` feeds forward expectations that harden thresholds as patterns repeat.

16.2 Impossible-state invariants (`attack_memory.rs invariants + model_checker.rs`)

- `StateInvariant + InvariantCheck` (`NeverWithCapability`, `NeverCoexist`, `NeverModified`, `MaxRisk`, `MaxCapabilities`, `Custom`) let the daemon encode **states that must never occur**, checked across tick.
- `BrokerModelChecker` (`model_checker.rs`) does `check_targeted()` exploration — a bounded model-checking pass (`ExplorationConfig/ExplorationResult/Violation`) over capability-broker state transitions.

16.3 Automata modeling (`automata.rs`)

Named fault-model DFAs (`privilege_escalation_automaton`, `data_exfil_automaton`, `persistence_automaton`, `network_c2_automaton`) are fed symbols per component; matching a full path raises `risk_score` and identifies the component. The `AutomatonManager` correlates components across automata (multi-stage attack recognition). Structure hashes make the learned automata auditable.

16.4 Adaptive thresholds & EWMA (brain.rs)

The brain's escalation thresholds are **adaptive** — sustained pressure tightens them (`adaptive_thresholds()`), keyword-signals massaged through an exponentially-weighted moving average. `record_enforcement_outcome` and `record_false_positive` re-tune behavior. When a kernel snapshot is present, `tick_with_kernel` folds judge signals into the decision, so the learning loop contains a verifiable, independent sanity chain.

16.5 The self-learning feedback loop (operational)

1. attack memory + threat graph → 2. autonomous escalation (eBPF drop, sandbox, **deception**) → 3. state hardening (thresholds/profiles/paranoid) → 4. feedback + re-baseline. Each probe both blocks *and* hardens. The loop is persisted and survives restart (JVC archive, HMAC), so machines *remember*.

17. The Memory System

17.1 The tiered threat-memory graph

A three-tier memory of observed threats with a **real binary wire format**: - **Hot tier** — last-N ThreatSignatures in-memory VecDeque (uncompressed, O(1) access). - **Warm tier** — compressed on-disk segments (JVC archive format). - **Cold tier** — compacted JVC archive (append-only full history). - **Elastic budget** — total bytes tracked; when hot exceeds budget the coldest entries (by access) evict to the compressed warm tier; hot size auto-tunes.

17.2 ThreatSignature — fixed 64-byte records

timestamp, threat_type, severity, src_ip, dst_ip, port, pid, meta_hash, flags (flags incl. `is_fast_blocked`). Fixed size makes XOR-delta + RLE compression possible (~3–5x on temporal locality of threat logs).

17.3 JVC (Jayce Vector Compression) wire format

`magic(b"JVC\x01") + version(4LE) + mode(1) + n_entries(4LE)`, then per entry `name_len(4LE) + name + compressed(1) + data_len(8LE) + data`. The `jvc_compressor_rt` crate provides the real engine (`jvx_codec.rs`, `entropy.rs`, `bsc.rs` Burrows-Scott-Crary entropy coder, `lz77.rs`, `sem/semantic.rs` semantic pass, `transcoder.rs`) — a multi-pass codec (JVX + LZ77 + entropy/semantic). `jvc_compressor/` holds the SDK stub types.

17.4 Windowed pattern memory & per-process intelligence

- **Pattern detection** — a sliding window of signatures is pruned and scored (`detect_patterns`, `ThemePattern`, `is_repeated(key, threshold)`).
- **Per-PID hazard history** (`history_for_pid`) — a running account of a process's past behavior, so old violations inform new decisions.
- **Batch risk scoring** (`batch_risk_score`) — scores entire PID sets at once; **engine detections** can be recorded into the graph (`record_engine_detection`).
- **Forecast & anchor** (`forecast`, `anchor`) — projected behavior and “anchored” stable-state markers.

17.5 What it powers

Threat recall, baseline/risk scoring, the forensic report's “attack memory” section, and the TUI's Memory-Graph panel. Because it is **append-only and HMAC-integrity-checked**, the memory doubles as an attack-evidence ledger: an attacker cannot rewrite what it “learned.”

18. The Adapters, Bridges & Shims

Every integration point is deliberate and capability-gated. This is the repository's stated “sanctioned ways in” (`AGENTS.md`).

18.1 Language adapters (`trust_terminal/.../adapter.rs`, `builtin.rs`)

A `LanguageAdapter` trait (`name`, `extensions`, `detect confidence`, `eval`, optional persistent sessions). Builtin adapters cover 19 languages — `python`, `javascript`, `typescript`, `go`, `ruby`, `lua`, `perl`, `php`, `swift`, `kotlin`, `csharp`, `haskell`, `rust`, `bash`, `java`, `c`, `cpp`, `zig` (+ `Apodeixis` as the first **non-spawning** adapter). Adapters translate inline code to external toolchains; the embedded `Apodeixis` adapter runs **in-process, deterministic, step-budgeted** (100k steps) — never a subprocess (subprocess `eval` is attack surface). Register new ones via `register_builtin_adapters`.

18.2 Semantic analyzer adapter (`trust_core/semantic_analyzer.rs`)

A pluggable `SemanticAnalyzer` (Box) is injected into the `PromptGuard`'s layer-3; future semantic backends can be swapped without changing the guard core.

18.3 GuardianAdapter (miniguard/src/guardian_adapter.rs, trust_core/guardian.rs)

A deterministic constraint gate between telemetry input and actuation: TelemetryFrame + ConstraintSets → ActuationVerdict (Permit / Delay / Reject), with a from_text line parser (action=... target=... kind=... signal.x=... label.x=...) so kernel rcf/kernel mission read the same format the REPL uses. **Deterministic (no I/O, no randomness), fail-closed on missing signals.** This is the “brain is cybersecurity, GuardianAdapter is physical safety” split.

18.4 Shroud adapter & shroud bridge

- **shroud_adapter.rs** — polymorphic outer layer translating host-OS semantics into shroud policy (path normalization file:/net:/proc:/reg:, permission-model mapping, process gathering). **User-mode only — no hooks, no drivers, no system modifications** (stays inside Windows/Linux ToS).
- **shroud_bridge.rs** — the daemon’s call surface into the shroud (PromptGuard + MemoryPoisonDefense + shroud state for the eval socket).

18.5 The Jayce SHIM — the frozen foreign ABI (jayce_kernel_src/src/kernel/shim.rs)

The kernel exposes **exactly five operations** to personalities/userland and “never hands out direct access to kernel internals”: send, recv, map_region, yield_now (plus log/attest/declare_use_class/require_policy/query_shape/query_compat/query_topology/require_core_boundary feature probes). Frozen at ABI v3 (JAYCE_SHIM_ABI_VERSION, freeze tag SHV1). Return codes SHIM_OK/INVALID/DENIED/FULL/EMPTY/NO_REGION. It carries capability-token checks on every call, and a SHIM_RIGHTS_STANDARD default capability set. Feature flags include SHIM_FEATURE_POLICY_GATE, SHIM_FEATURE_ATTESTATION, SHIM_FEATURE_COGNITIVE_STATE, ... and — critical for the “no-backdoor/civilian-only” contract — **SHIM_FEATURE_CIVILIAN_USE_ENFORCED**: declare_use_class only accepts SHIM_USE_CLASS_CIVILIAN; military/weapons use classes are **denied at the ABI boundary** (SHIM_USE_CLASS_MILITARY_DENIED = 0xFF_DEAD).

18.6 The six-step request pipeline (shim_flow.rs)

Typed FlowStep enum codifies the exact OS personality → kernel → back path, with ShimRequestKind and ShimValidationResult (validate_request), IllusionLayerId (positioning illusion evaluation in the flow), and a self-verifying verify()/all_pass() report of the pipeline’s contractual order. This is the “flow” every personality request is proven against.

18.7 Daemon bridges (the layered bridges)

- `kernel_bridge.rs` — daemon ↔ canonical kernel cell (lock-guarded).
 - `trust_terminal_bridge.rs` — `term:<line>` ↔ SmartTerminal (Prompt-Guard first).
 - `mg_ai_bridge.rs` — `KernelAiBridgeClient` AI-telemetry frames.
 - `cosmic_bridge.rs` — telemetry pipe → Cosmic Map.
 - `shroud_bridge.rs` — daemon ↔ shroud content-scanners.
 - `cohost.rs` / `cohost_compat.rs` — co-host OS integration.
-

19. Clustering & Fleet Topology

19.1 Kernel cluster topology (`jayce_kernel_src/.../cluster_topology_adapter.rs`, `host_cluster.rs`)

Kernel-side peer registry + browsing: `register_peer`, `deregister_peer`, `send_topology_ipc`, `propagate_health`, `query_quotas/query_status` (`ClusterQuotas`, `ClusterStatus`, `ClusterCallResult`). The host engine re-exports a host-addressable subset for the daemon's `kernel cluster` command.

19.2 Gossip transport (`miniguard/src/gossip_transport.rs`)

TLS-mandatory, mutual-auth, HMAC-SHA256, chain-hashed, per-peer rate-limited peer messaging (also subject to the Apodeixis connection-integrity gate, §10). `peer_states` with a 10-min TTL feed the Cosmic Map's fleet view.

19.3 Cosmic Map role mapping

Fleet UI maps threat level → status: `Calm=online`, `Suspicious=degraded`, `Hostile/Quarantined=offline-red`. The daemon (node 1) emits `JAYCE_TLM`: frames with every peer; `miniguard --map` renders without root. Includes a `cluster` module, `multiverse` (up to 16 forked kernel-universe instances with identity/metadata/lifecycle) in the kernel, and `master_console` (the cloud brain-swarm replication: registry, immunity, telemetry, client/server).

19.4 Master Brain / swarm immunity (roadmap)

An **opt-in** aggregate: nodes upload anonymized attack signatures (hash of payload entropy + behavior graph + IP heuristics, zero PII) to a Master Brain, which validates and pushes back signed OTA immunization rules (Ed25519 + HMAC chains + Apodeixis proof contracts). Air-gapped/offline deployments stay 100% local (§11, §9.7).

20. The Illusion Shroud

The shroud is the **user-mode enforcement overlay** that translates host-OS semantics into a ring/capability policy and actively defends content-level and peripheral surfaces. It is deterministic, user-mode only, and kill-switchable.

20.1 The ring model (`shroud.rs`)

Resources are scoped to a **Ring** (System / File / Network / Process / User / Admin) with per-ring capabilities, grants, and budgets (`RingBoundaryInfo` documents each: System = “ring-zero isolation”, File = scoped read/write, Network = allowlisted outbound, Process = spawn/kill within sandbox, User = interactive access / UI / clipboard, Admin = deny-by-default for dangerous ops). Ops ask `evaluate(resource, ring, context) → PolicyDecision`; the shroud can `activate(pid)`, `deactivate`, `emergency_stop`, `audit_only`, `grant/revoke` capabilities, `filter_output`, `chained-audit(verify_chain)`, and `exportJSON` audits. A global `shroud_set_active` master switch.

20.2 The five hardening layers (`shroud_harden.rs`)

1. **Integrity Verification** — hash tracked files (`IntegrityEntry` known-good hash, `IntegrityVerifier`, auto-verify every 60 s) → tamper detection.
2. **Canary Tokens** — honeypot files that alert when touched.
3. **Behavioral Profiling** — baseline normal behavior, alert on deviation.
4. **Dead Man’s Switch** — self-monitoring + auto-recovery if the shroud fails.
5. **Response Playbook** — automated escalation (revoke → kill → quarantine).

20.3 Network shield + firewall (`shroud_network.rs`, `shroud_firewall.rs`)

- **Network Shield** — a DDoS classifier / threat detector over the five shroud counters; reports `VoidPitStatus` (void-pit-isolated sources).
- **Firewall** — the *enforcing* half: egress + ingress filters evaluated by a top-down rule engine (first match wins), **default DENY**. Blocks blacklisted IPs/CIDRs, void-pit sources, port-scanners (sequential probes), rate-exceeding traffic, unauthorized outbound, and payload attack signatures.

20.4 Peripheral & wireless guards (`shroud_peripheral.rs`, `shroud_wireless.rs`)

- **Peripheral** — USB / HID device policy and alert/block counters.
- **Wireless** — rogue-AP / deauth attack detection and block counters (`shroud_wireless_*`).

20.5 Crypto & service (shroud_crypto.rs, shroud_service.rs)

- **shroud_crypto** — cryptographic primitives used by the shroud.
- **shroud_service** — the shroud as a serviceable overlay.

20.6 The shroud as adapter-adjacent state (§9, §18)

The daemon bridges into the shroud's PromptGuard, MemoryPoisonDefense, and hardening layers for the AI-content pipeline and the eval socket. The kernel mirrors the shroud counters in `illusion.rs` `ShroudMetrics` (§5.3) so the fleet can see shroud posture remotely.

21. The Kernel Illusion Layer & the Frozen ABI

The kernel-side defense is the *deterministic* complement to the daemon's user-mode engine: - **Dual-Layer Illusion** (§5.2) masks timing / addresses / identities / statuses / events for probes under `IllusionState` pressure. - **Five-operation SHIM** (§18.5) — the only foreign ABI; capability-token checked; **civilian-use-class enforcement at the boundary** (weaponization refused). - **Capability tokens** (capabilities/mod.rs) — every shim call takes a `token_id`; rights are revocable, time-bounded bitmask grants (`CAP_IPC_SEND/RECV`, `CAP_MEMORY_ALLOC`, `CAP_NETWORK_TX/RX`, ...). - **doorman / constitutional_shadow_layer / burn_pit / void_pit / impossible_state_detector / temporal_proof_ledger / stack_canary / cold_boot_defense / iommu_policy / pci_config** — per-surface hardened kernel engines (~80 total). - **boot_trust.rs** — verifies the boot chain before mainstream engines run. - **spine/invariants.rs** — the immutable Spine: 40+ declared engines in `ENGINE_BUDGETS` (22 active in the security budget), endpoint-invariance hash (FNV-1a32 "SPIN") to prove the kernel hasn't drifted. - **upgrade.rs / peace_audit.rs** — kernel-internal upgrade and audit paths.

These are the "hardened surfaces" of the kernel proper: a reverse engineer hitting shims gets capability denials + use-class denial + illusion-masked answers, all deterministic.

22. The TRust Cognitive Terminal

The terminal is not a REPL — it is a **cognitive surface** that reasons about code, sessions, and the kernel. Components:

22.1 SmartTerminal core (`terminal/smart_terminal/mod.rs`)

Holds a LanguageRegistry, SessionManager, Linter, Plumber, and Bloodhound. Main entry process `input(input, cwd) → TerminalResponse` routes every line through intent parsing (`intent.rs`), Prompt-Guard-style guarding, inline-language `detect_language/eval/eval_file`, command completion (`suggest_commands`), and a command-reference catalog (`command_reference.rs`).

22.2 Inline evaluation across 19 languages + Apodeixis

`lang run <file|code>`; the embedded Apodeixis adapter is the first **non-spawning** adapter — deterministic, proof-checked, step-budgeted (§18.1).

22.3 Static analysis / linting (`linter.rs`)

`linter <file|code>` — a static **security + quality taint scanner** across many languages: `LintReport` with severity classes (error/warning/info/hint), syntax checking, plus `lint_file / lint_code / lint_dir` (recursive) / `lint_and_format`.

22.4 Plumber (`plumber.rs`)

`plumber <dir|file>` — an architecture/code-quality auditor that grades code into `IssueSeverity` (Critical / Major / Minor / Cosmetic) with `auto_fixable` counts — finds dead pub fns, suspicious patterns, and structural issues.

22.5 Bloodhound — dead-code & threat tracing (`bloodhound.rs`)

`bloodhound / sniff` — a recursive cross-file tracer that builds a **definitions→references graph** and classifies unreachable code into a `DeadKind` taxonomy (corpses, ghosts, suspects), producing a `DeadCodeReport` (`is_clean / summary / by_kind`). This is the “diagnostic sniffer” used to spot shadow/unreachable surfaces that attackers rely on.

22.6 Diagnostic assistant (`terminal/debug_assistant/`)

Stack-trace analytics for crashes: `stack_parser`, `graph_tracer`, `smell_detector` (code smells / OWASP-adjacent patterns), `analyzer`, and `report` — a mini root-cause engine for anomalous/failed processes.

22.7 Sessions, aliases, help (`session.rs`, alias handling)

Persistent language sessions (`start_session`, `eval_in_session`, `stop_session`), user aliases, `help`, and a large command catalog.

22.8 Kernel subcommand surface (`kernel_cmd.rs`)

The operator's window into the whole engine: `kernel status`, `forensic/forensics/report` (the forensic dump), `logs`, `truth`, `token` (doorman), `vitals`, `heal`, `cluster`, `services`, `monitor`, `burnpit`, `illusion`, `shadow`, `isd`, `pit` (void pit), `net`, `net-block/net-unblock/net-inspect`, `piggyback`, `watchdog`, `spine`, `tpl`, `universe` (`create/fork/delete/list/get`), `rcf`, `mission`, `saddle`, `vm`, `robot/drone/vehicle/swarm`, `bridge`, `signal`, `environmental`, `home`. Named Apodeixis missions resolve here (`robot-*`, `auth`, `valid-json`, `safe-update`).

22.9 Why it's part of security

The terminal is **the** human-facing control plane: it enforces Prompt-Guard before any term: input, holds the forensic dump, gates kernel actuation through GuardianAdapter + Apodeixis, and exposes live deception/void-pit/TPL state. A compromised terminal is caught by the daemon's fail-closed guard, not treated as an open channel.

23. Watchdog, Sentinel, Gates, Seal & "No Backdoors"

23.1 Watchdog (`watchdog.rs`)

An independent supervisor (forked child protected by `PR_SET_PDEATHSIG`) that closes the **"kill -9 miniguard" gap**: it watches the main process via `waitpid` + heartbeat file (`/run/jayce/miniguard.heartbeat`; stale > 30 s ⇒ restart) and restarts it within seconds. WatchdogConfig bounds restarts, delay, and heartbeat timeout (default infinite restarts).

23.2 Antifreeze sentinel (`sentinel.rs`, `bin/sentinel.rs`)

The **external defender a frozen daemon cannot be** (`SIGSTOP` blinds anything inside the daemon). Every 2 s: reads `/proc/<daemon>/status`; State: T or a stale heartbeat ⇒ **immediate SIGCONT**; snaps evidence + culprit root shells to `freeze_events.log`; escalation ladder ≥ 3 freezes / 60 s ⇒ `SIGKILL` for a clean systemd restart. It heartbeats itself — the daemon watches the sentinel too (mutual watch; a dead sentinel in an attack window is itself signal; the daemon restarts it after 120 s cooldown).

23.3 The gates (every pathway is gated)

- **Shim gate** — capability token + use-class on every shim call (§18.5/§21).
- **Doorman** — kernel capability-token verification/authorization for requests.

- **GuardianAdapter gate** (§18.3) — deterministic physical-safety allow/delay/reject.
- **Prompt Guard / TRust guard** — every term: line gate, fail-closed (§9.4).
- **Admission gate** (`admission.rs`) — pre-ingress trust re-verification.
- **Update gate** (§10.4) — safe-update Apodeixis mission vetoes OTA install.
- **Boot gate** (`boot_trust.rs`) — verifies boot chain before engines run.
- **Personality gate** (`shim personality_boot_gate_denied_count`) — bogus personalities are denied at boot.

23.4 Seal — the no-tamper boundary (`seal.rs`, `scripts/seal.sh`)

Immutable binaries + units + config (`chattr +i`), hash manifest (`seal.hashes`), read-only `/usr/local/bin` bind mount, kernel lockdown, and runtime cross-verification (§2.4). The seal is the “you cannot undo the hardening” guarantee.

23.5 No-backdoor guarantees (structural)

- **Civilian-use-class enforced at the ABI** (`SHIM_USE_CLASS_MILITARY_DENIED`).
- **No hidden escape via logging/telemetry** — observability is socket-only; there is no journald/telemetry side-channel to exfiltrate through (§11, §9.7).
- **Append-only, HMAC-verified audit + memory** — history can’t be rewritten to conceal an earlier compromise (§3.7, §17).
- **`no_ambient_trust.rs`** — nothing trusted by default; trust is earned and revocable.
- **`instance_lock.rs`** — single-instance enforcement prevents forked duplicates racing the daemon.
- **`licensing.rs`** — enforces the proprietary source-available license so the hardened build can’t be silently relicensed/repurposed.
- **`keyring.rs`** — secrets live in a guarded keyring, not hardcoded.
- **No self-lockout trapdoors** — `self_identity` never self-enforces; the paranoid “sustained” gate prevents accidental lockouts; the sentinel’s seal-violation path is one-shot (no endless kill loop). The fail-closed paths are deliberate and observable, not hidden.

24. The Metrics TUI & Operational Console

`miniguard-status` (the TUI `miniguard-status`, plus the `saddle` bin) renders the live daemon state over the token-authenticated socket. It is the “lists all metrics” surface.

24.1 Pages & navigation

- **Page 0 — Main dashboard** (`render_dashboard`), with **live panel focus** (`←/→` cycle focus across `PANEL_COUNT` panels; `↑/↓` scroll a focused panel).
- **Page 1 — AI Safety** (a/A toggles; `render_ai_page`) — process containment on top, content-level (memory-poison) below: `render_ai_self`, `render_ai_memory`, `render_ai_containment` (the envelope tag map).
- **q / Esc** quits; **c / C / r / R** clears the daemon audit/ring.

24.2 Main-dashboard panels

`render_header` (Seal line: kernel lockdown / Secure Boot (SB=on/off) / TPM / seal manifest / daemon+sentinel immutability), engine status, **DDoS, network + process, threat feed** (color-coded by threat level — quarantined = magenta), **telemetry, memory graph, audit trail**, top/bottom rows, and a footer. Visual languages: `gauge_bar`, `risk_bar`, `jitter_color` (tick jitter latencies), `illusion_color` (deception state), scenarios (calm / suspicious / hostile / ddos / paranoid) for demo/self-test.

24.3 What's surfaced

Threat level + scenario, threat feed, engine/tick health + jitter, network + process monitor, DDoS counters, memory-graph tier stats, the hash-chained audit trail, enclave/illusion state, AI containment tags (`fs:yes` vs `fs:no`), AI memory threats, and self-integrity metrics. `fetch_status` deserializes the daemon's status JSON into `DashboardState`; `clear_daemon_audit` resets the audit ring over the socket.

24.4 The saddle console (`bin/saddle.rs`)

The robotics operator variant: units table + `SaddleMap` + actuation ledger + kernel/judge TPL panes + command bar, with `--mini` and `--touch` evdev gesture bridging (§12.4).

25. How It All Interlocks

MiniGuardian is one organism, not a bag of features. Trace one attack and every layer you asked about appears:

1. **Probe hits a surface** → the kernel shim returns a capability-denied / use-class-denied / illusion-masked answer (§21) and the shroud firewall DROPs or the illusion bubble **deceives** (§5).
2. **The signal lands** → `ScanDirector` (`main.rs`) triggers a detector (§3); the incident becomes a `BrainEvent` with an `EventSource` tag.

3. **The brain learns** → `attack_memory / memory_graph` record it as a `ThreatSignature` into the hot tier, mirrored to compressed warm/cold JVC tiers (§17), scored against automata + impossible-state invariants (§16).
4. **The brain escalates** → brains talks to the **kernel judge** through `agreement.rs`; together they lift `Suspicious` → `Hostile` → `Paranoid`, which activates eBPF/IPtables enforcement (§4), deception expansion, and threshold tightening.
5. **The operator sees it all** → `kernel_forensic`, the `miniguard-status` TUI (§24), and the Cosmic Map render the lesson, the deception observations, and the seal posture.
6. **If the daemon dies or freezes** → the watchdog restarts it, the `sentinel` `SIGCONT/SIGKILLs` (§23), and the persisted memory re-anchors on restart — the machine *remembers* why it hardened.
7. **AI tools** live inside their own wall: `PromptGuard` + memory-poison defense
 - `eval socket + envelope` (§9), so the model is fed only through the deterministic scanners — deceive the model is possible, deceive the wall is not.

The unifying idea (repeated in the docs): **“the AI/model/attacker is not the trust boundary — the deterministic container around it is.”** Every subsystem above is that container, extended across process, kernel, memory, network, deception, and fleet.

26. The Biggest Illusion: Self-Disguise & Probe-Feed

The deepest deception is **the daemon hiding itself** and — when someone does find and probe it — **answering every probe with fabricated, internally-consistent falsehoods**. Two distinct mechanisms:

26.1 Self-disguise: the daemon is not “miniguard” (`process_mask.rs`, `main.rs`)

At startup (`main.rs` → `mask_as_kworker()`), the daemon renames its process name and clobbers its `argv` so it does **not** look like a security daemon: - **`mask_process_name`** writes `/proc/self/comm` to a `kworker`-style name (`kworker/0:1`, `kworker/u8:2`, `kworker/1:0`, `kworker/0:1H` — stored up to the 15-byte `comm` limit, chosen deterministically by PID). - **`set_process_title`** calls **`prctl(PR_SET_NAME)`** so even advanced `ps/top` tooling sees a worker-thread persona, not miniguard. - Combined with the **`argv/envIRON scrub`** (§2.1): `/proc/self/cmdline` and `/proc/self/environ` are empty. A process-table scan shows a nameless `kworker/N:M` thread with a blank command line — indistinguishable at a glance from a kernel worker. An attacker listing “suspicious daemons” simply does not see one. - **Why it’s honestly called an illusion**: the source documents that root can still locate the daemon via `/proc/self/exe`, and the trust engine therefore

never identifies the daemon by process name (`self_identity` uses `pid + exe + cgroup` precisely because a masked name is spoofable). The disguise is the *first* layer — it forces deeper probing (`exe/cgroup`), which is exactly where the probe-feed (§26.2) and the anti-analysis detector (§2.2) are waiting.

26.2 Probe-feed: when probed, it feeds false information

Two cooperating bubble systems, both on the same principle — **give a plausible lie, never silence** (silence itself signals that you're defended):

(A) Kernel Illusion Bubble — the synthetic-hardware world

(`jayce_kernel_src/src/kernel/engines/illusion_bubble.rs`) Each OS personality runs inside a sealed bubble presenting a **fake, internally-consistent hardware projection** (`SyntheticHardwareProfile`): fake CPU (CPUID answer), fake physical-memory base + **bounded** content size (never reflects real RAM), fake PCI vendor, fake APIC base, fake timer base (TSC origin), fake IRQ mask, fake ACPI OEM — the guest *never touches real hardware*. - **Stateful**, so read-after-write on fake registers is consistent — a necessary condition to look real to sophisticated attackers probing for interception. - **Rotates every 64 ticks** (CPU-ID drift, IRQ-mask shift, timer-base noise, deterministically derived from tick + seed) so the same fingerprint is never seen twice — defeats persistent cross-session fingerprinting. A FNV-1a `profile_hash` lets the kernel verify the projection wasn't tampered.

When anything in that synthetic world is **probed or attacked**, `intercept()` → `adjudicate()` returns one of five answers:

Probe / action the attacker performs	What the attacker receives
ReconProbe (CPUID, ACPI, device enum), TimingProbe	SyntheticNoop — no response at all; recon starves for signal
HardwareWrite, DmaHardwareDirect, InterruptManipulation, DriverInstall	SyntheticSuccess — absorbed, a <i>plausible success</i> returned; nothing real happens; prevents retry-escalation
RawSocketAccess, PrivilegeEscalation, MemoryEscape, RawDiskAccess	SyntheticFailure — absorbed, a <i>permission-denied</i> returned; nothing real happens; no “you caught me” leak
FirmwareAccess, KernelPatch, CrossBubbleAccess, BootChainWrite	Quarantine — bubble isolated; MMU address-space domain destroyed at the hardware level so future context switches fault
HardwareRead / real vNIC traffic	Passthrough — genuine, mediated I/O (the user keeps real internet; only transport-layer <i>escape</i> patterns are watched)

Escape attempts are counted; ≥ 32 **crossings auto-quarantine** the bubble. A Frozen bubble silently returns `SyntheticNoop` (no escalation signal at all). Each of the 6 personalities maps to its own bubble slot bound to a **minimal PERSONALITY_CAP_MASK** (only what it needs) — a compromised guest lives inside a convincing lie with the narrowest possible real privilege.

(B) User-mode Illusion Bubble — fake command / honeypot layer

(`miniguard/src/illusion_bubble.rs`, §5.1) The daemon's command gate does the same at shell level: dangerous commands are fed **fake but plausible output** (`cat /etc/shadow`, `id`, `whoami`, `uname`, `ifconfig`), and the fake files on disk carry **MG_HONEYTOKEN_* markers**, so when a stolen value comes back through any channel, that is *confirmation* of theft (§5.1).

26.3 Why feeding falsehoods is stronger than blocking

- **No detection feedback.** A deny tells the attacker “I’m watched.” A plausible lie withholds that — recon is consumed without confirming a defender exists.
- **Time burn.** The attacker builds exploit chains on a fake CPU, fake memory map, fake ACPI — studying hardware/OS that isn’t real.
- **Confirmation trampoline.** The fake world is *consistent* (stateful, rotating, hash-verified), so the attacker can’t tell “real” from “lie” — and the credentials they exfiltrated are the honeypot markers that eventually prove the theft.
- **Quarantine is the lie that ends.** The moment a true boundary crossing happens (firmware, kernel patch, boot chain, cross-bubble), the lie collapses into **hard isolation** — MMU domain destruction — so the deception never rewards an actual breakout.

Codebase principle: *deceive the model/attacker is possible, deceive the wall is not*. §26.2(A) is the wall’s answer to a probe — and §26.1 is the wall hiding so the probe never even knows to aim.

27. The Alien Bridge, Foreign-Sensor Ingestion

27.1 What it is

The **Alien Bridge** (`jayce_kernel_src/src/kernel/engines/host/alien_bridge.rs`, `canonical ../engines/alien_bridge.rs`, daemon wrapper `miniguard/src/alien_bridge.rs`, re-exported as `jk::engines::host::alien_bridge`) is the kernel’s deterministic “**foreign signal → semantic meaning**” seam. It is the sanctioned ingestion boundary for data that does **not** originate in the native kernel: external sensors, vision/LASV frames, other OS personalities, raw field/devices. Foreign input is normalized into a canonical 0–65535 amplitude space and translated into **typed, capability-**

bounded, unit-annotated SemanticObservations — never exposed as raw datums with privilege.

27.2 Two halves, one surface

- On the **bare-metal kernel**, the host engine delegates to the canonical `crate::engines::alien_bridge`.
- On a **host OS** (the daemon case), it runs a deterministic, **bounded in-process ring** (16 recent observations, ≤ 8 labels of 32 bytes) that normalizes raw samples into `SemanticObservations`. All kernel state under `KernelCell / jk::with_kernel_lock`; atomic counters; no heap — foreign input cannot perturb the deterministic core.

27.3 The raw → semantic pipeline

Each sample is typed through a ladder that turns an amplitude into a `SemanticObservation`: - **Raw signal types**: `DigitalPulse`, `DigitalBurst`, `AnalogWaveform`, `AnalogSpike`, `Packet`, `FieldFluctuation`, `NoiseOnly`, `Unknown`. - **Pattern detection**: `Constant`, `Periodic`, `Burst`, `Transition`, `TriggeredPulse`, `Envelope`, `Anomaly`, `Unknown`. - **Abstract action**: `Trigger`, `StateChange`, `DataStream`, `PresenceDetect`, `Activation`, `Deactivation`, `ModeToggle`, `Anomaly`, `Unknown`. - **SemanticObservation** = `SignalCapability` (`ButtonPress`, `PresenceDetect`, `MotionPattern`, `DataStream`, `ModeToggle`, `BioRhythm`, `ProximitySense`, `ActivationSignal`, `UnknownDevice`) + `SemanticMeaning` (`UserIntent`, `PresenceState`, `MotionState`, `EnvironmentalLevel`, `StructuredPayload`, `RhythmState`, `ActivationState`, `ModeState`, `FaultState`, `Unknown`) + `ObservationUnit` unit discipline (`None`/`Boolean`/`RelativeLevel`/`SampleMagnitude`/`FrequencyHz`/`PulseCount`/`EncodedWord`/`Confidence`) + `value`, `confidence` (0–255), `urgency` (0–255), `provenance`, `explanation_code`, `timestamp_us`. - **Amplitude-band classification** (`classify_amplitude`) maps raw magnitude to (`capability`, `meaning`, `unit`, `confidence`, `urgency`) — e.g. 0–15k → `UnknownDevice` (low conf), 15–30k → **`PresenceDetect/PresenceState`**, 30–45k → **`ActivationSignal/ActivationState`**, 45–60k → **`MotionPattern/MotionState`**, $\geq 60k$ → **`FaultState`** (highest confidence/urgency). - **Explicit translations** can bypass classification via `record_translation`, and `record_sample/last_observation/recent_observations/status_summary` expose the ring to the daemon's telemetry.

27.4 The daemon wrapper (`miniguard/src/alien_bridge.rs`)

- **`init()`** — enables + resets the kernel bridge, labels the source "miniguard-host", and `ensure_personalities()` registers two **virtual device personalities** (`AlienDevice`, `ForeignOS`) so a multiverse universe with one

of those profiles can consume the signal stream “as if it were a physical device.”

- **record_host_sample(tick, threat, cpu_pct)** — synthesizes a deterministic amplitude from daemon runtime state (tick, threat level, CPU %), clamps ≤ 65000 , timestamps wall-clock micros, records into the kernel, and posts DataReady events to the alien/foreign personality slots. `tick()` calls this every tick.
- **status_summary()** — a telemetry/UI snapshot (AlienBridgeStatus: enabled, source/observation/label counts, last capability/meaning/confidence/urgency).
- **register_editor_personality(binary, pid)** — registers a detected AI editor (VS Code, Zed, Cursor, Devin...) as a virtual ForeignOS personality, **bridging the AI-containment detection (§9) into the kernel's semantic domain.**

27.5 Why it's a security feature

The alien bridge is the **choke-point for all foreign sensor input**: foreign data must cross this deterministic translation **and** (in the LASV path, §12.3) a SensorForge mission-verified, taint-aware gate **before** it can influence the actuation path (PathSense clearance / fused risk). It turns an unbounded, untrusted foreign signal stream into a small, typed, capability-attributed vocabulary with unit discipline and confidence — so a hostile or poisoned sensor feeds *semantic observations into a constrained grammar*, not raw control into the kernel. Its determinism + boundedness + kernel-lock discipline make it a safe seam between the unpredictable real world and the provable infrastructure (§8, §14).

End of catalog and deep dive. Operational run-books: AGENTS.md. Threat-model peer review + known caveats: docs/security_assessment.md, §7.4 (Apodeixis proof-checker unsound edges), §7.3 (container mode privilege posture). AIE spec + risk gate: docs/AI_WORKSPACE_ENVELOPE_SPEC.md and docs/AIE_IMPLEMENTATION_RISK.md.